

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
ÁREA PROFESIONAL

Software Avanzado

Docente: Ing. Moisés Granados



Informe Técnico: Comparativa de Proveedores de Nube con PoC

Carné:

201430801

Nombre:

Luis Basilio Baquiaux Sic

Informe Técnico: Comparativa de Proveedores de Nube con PoC

**Práctica: Despliegue de UniMarket CUNOC en la
Nube**

Contenido

Informe Técnico: Comparativa de Proveedores de Nube con PoC.....	1
Introducción.....	4
1. Fundamentos de la computación en la nube relevantes al caso.....	5
2. Mapeo de Servicios Equivalentes.....	6
3. Análisis por Dimensiones Técnicas.....	8
3.1 Precio y Modelo de Costos.....	8
3.2 Seguridad.....	9
3.3 Velocidad y Rendimiento.....	10
3.4 Experiencia de Usuario para Desarrolladores (DevEx).....	10
3.5 Integración con Lenguajes de Programación y Frameworks.....	11
3.6 Disponibilidad de Herramientas y Ecosistema.....	12
4. Pruebas de Concepto (PoC).....	12
4.1 Stack Tecnológico Elegido y Justificación.....	12
4.2 PoC en Amazon Web Services (AWS).....	13
4.3 PoC en Google Cloud Platform (GCP).....	15
4.4 PoC en Microsoft Azure.....	19
4.5 PoC en DigitalOcean.....	23
4.6 Comparativa Empírica.....	26
5. Recomendación Fundamentada.....	27
5.1 Ventajas y Desventajas por Proveedor.....	27
5.2 Recomendación Técnica.....	28
6. Reflexión Crítica.....	30
6.1 Transferibilidad de habilidades entre proveedores.....	30
6.2 Riesgos del vendor lock-in.....	31
6.3 Estrategia multi-nube o híbrida.....	31
6.4 Contraste entre teoría y práctica.....	32
7. Conclusiones.....	34
8. Referencias.....	35

Introducción

La computación en la nube ha consolidado el modelo predominante para el despliegue de sistemas de software a escala global. Sin embargo, la oferta de proveedores es amplia y heterogénea; cada plataforma presenta diferencias significativas en costos, modelos de seguridad, rendimiento, experiencia de desarrollo, ecosistema de servicios gestionados y compatibilidad con lenguajes de programación.

La selección de un proveedor de nube no es una decisión meramente comercial: condiciona la arquitectura del sistema, el presupuesto operativo, la postura de seguridad y la velocidad de entrega del equipo de desarrollo. Por ello, el ingeniero en sistemas debe ser capaz de evaluar objetivamente las alternativas disponibles antes de comprometer recursos y esfuerzo técnico en una plataforma específica.

El presente informe aplica ese principio al proyecto **UniMarket CUNOC**, ya planificado y estructurado bajo marcos ágiles (Scrum y Kanban) en la práctica anterior. En esta etapa, el sistema debe trasladarse a un entorno de producción en la nube, y el equipo de desarrollo aún no ha definido qué proveedor utilizará. Este documento tiene como propósito realizar un análisis comparativo técnico y empírico de cuatro proveedores de infraestructura en la nube: **Amazon Web Services (AWS)**, **Google Cloud Platform (GCP)**, **Microsoft Azure** y **DigitalOcean** como cuarto proveedor de libre elección.

La elección de DigitalOcean como cuarto proveedor se justifica por las siguientes razones: su propuesta de valor está orientada explícitamente a desarrolladores individuales y equipos pequeños, con una consola simplificada, precios predecibles sin costos ocultos por egress de datos dentro de ciertos límites, y una curva de aprendizaje significativamente menor que los tres grandes proveedores. Para un equipo universitario con recursos limitados, evaluar esta alternativa resulta pertinente y representativo de la realidad de muchos proyectos de software en etapas tempranas.

1. Fundamentos de la computación en la nube relevantes al caso

La computación en la nube se organiza comúnmente en cuatro modelos de servicio, que se diferencian por cuánto de la infraestructura administra el proveedor y cuánto queda bajo responsabilidad del equipo de desarrollo. Comprender esta distinción es relevante para el caso de UniMarket CUNOC porque cada componente de su arquitectura de referencia (microservicios, base de datos, funciones eventuales, API Gateway) se ubica en un punto distinto de este espectro, y las decisiones de despliegue tomadas durante los cuatro PoC de esta práctica corresponden directamente a estos modelos.

IaaS (Infrastructure as a Service) es el modelo donde el proveedor entrega la infraestructura de cómputo básica (máquinas virtuales, redes y almacenamiento en bruto) y el equipo de desarrollo es responsable de instalar el sistema operativo, el runtime, las dependencias y el propio software. Es el modelo con mayor control y también con mayor responsabilidad operativa. En el caso de UniMarket CUNOC, ningún componente del PoC se desplegó directamente sobre este modelo (no se usó, por ejemplo, una instancia EC2 o Compute Engine sin gestionar), pero es la capa sobre la cual están contruidos, de forma transparente para el usuario, todos los servicios PaaS que sí se utilizaron.

PaaS (Platform as a Service) es el modelo donde el proveedor administra además el sistema operativo, el runtime y el escalado, dejando al equipo de desarrollo únicamente la responsabilidad de su código y su configuración. Este fue el modelo predominante en los cuatro PoC de esta práctica: Cloud Run (GCP), Azure Container Apps, DigitalOcean App Platform y Amazon ECS Express Mode son todos servicios de este tipo, específicamente, una variante de PaaS orientada a contenedores conocida como CaaS (Containers as a Service), donde el equipo solo entrega una imagen de Docker y el proveedor se encarga de ejecutarla, exponerla mediante una URL pública, y escalarla según demanda. Las bases de datos gestionadas utilizadas (Cloud SQL, Azure Database for PostgreSQL, DigitalOcean Managed Database y Amazon RDS) son también ejemplos de PaaS: el equipo no administra el sistema operativo del servidor de base de datos ni aplica sus parches de seguridad, solo se conecta a un endpoint y gestiona su esquema y sus datos.

SaaS (Software as a Service) es el modelo donde el proveedor entrega una aplicación completa y funcional, lista para usarse a través de un navegador o una API, sin que el usuario gestione ni siquiera el código de la aplicación. Aunque UniMarket CUNOC no despliega ningún componente propio como SaaS (es, en sí mismo, la aplicación que se está construyendo), el proceso de esta práctica sí dependió de herramientas consumidas bajo este modelo: GitHub como plataforma de control de versiones y Docker Hub como registro de imágenes, son ambos servicios SaaS de terceros que se integraron en el flujo de despliegue sin requerir infraestructura propia para operarlos.

FaaS (Function as a Service), también conocido como *serverless* en su forma más pura, es el modelo donde el proveedor ejecuta fragmentos de código en respuesta a eventos específicos, sin que exista un servidor persistente que el equipo deba mantener corriendo; el cómputo se aprovisiona únicamente durante la ejecución de la función y se factura por invocación. Este modelo es directamente relevante para el requerimiento de UniMarket CUNOC de procesar tareas eventuales, como el envío de notificaciones o validaciones asíncronas, para las cuales los servicios equivalentes de cada proveedor (AWS Lambda, Google Cloud Functions, Azure Functions) representan la opción arquitectónica más

adecuada frente a mantener un servicio de cómputo corriendo permanentemente solo para tareas ocasionales.

2. Mapeo de Servicios Equivalentes

Para cada uno de los cuatro proveedores evaluados se identificó el servicio equivalente a las capacidades de referencia definidas en el enunciado. La tabla siguiente presenta el mapeo completo con nombre del servicio, tipo de servicio y descripción funcional.

Capacidad	AWS	GCP	Azure	DigitalOcean
Computación (EC2)	EC2 (IaaS) Instancias de máquinas virtuales escalables bajo demanda con soporte para múltiples familias de hardware.	Compute Engine (IaaS) VMs configurables con opciones de preemptible para reducir costos en cargas tolerantes a interrupciones.	Azure Virtual Machines (IaaS) Instancias de VM con integración nativa con Active Directory y servicios de identidad corporativa.	Droplets (IaaS) VMs de propósito general con precios fijos y predecibles, orientadas a simplicidad operativa.
Almacenamiento de objetos (S3)	Amazon S3 (PaaS) Almacenamiento de objetos con alta durabilidad (11 nueves), versionado, ciclos de vida y políticas de acceso granulares.	Cloud Storage (PaaS) Almacenamiento de objetos con clases de almacenamiento diferenciadas por frecuencia de acceso (Standard, Nearline, Coldline, Archive).	Azure Blob Storage (PaaS) Almacenamiento de objetos con integración nativa en el ecosistema Microsoft y soporte para capas de acceso (Hot, Cool, Archive).	Spaces (PaaS) Almacenamiento de objetos compatible con la API de S3, con CDN integrado opcional y precios planos por capacidad y transferencia.
Base de datos gestionada (RDS)	Amazon RDS (PaaS) Bases de datos relacionales gestionadas con soporte para MySQL, PostgreSQL, MariaDB, Oracle y SQL Server. Incluye Multi-AZ y réplicas de lectura.	Cloud SQL (PaaS) Servicio gestionado para MySQL, PostgreSQL y SQL Server con escalado automático y alta disponibilidad configurable.	Azure Database for PostgreSQL / MySQL (PaaS) Servidores gestionados con opciones Flexible Server para mayor control de configuración y zona de disponibilidad.	Managed Databases (PaaS) Servicio gestionado para PostgreSQL, MySQL, Redis y MongoDB, con backups automáticos diarios y failover automático.
Serverless (Lambda)	AWS Lambda (FaaS) Ejecución de funciones en	Cloud Functions (FaaS) Funciones serverless	Azure Functions (FaaS) Plataforma de funciones	Functions (FaaS) Servicio de funciones serverless

Capacidad	AWS	GCP	Azure	DigitalOcean
	respuesta a eventos con soporte para Node.js, Python, Java, Go, .NET, Ruby y runtimes personalizados.	orientadas a eventos con soporte para Node.js, Python, Go, Java, Ruby, PHP y .NET.	serverless con integración nativa con servicios Azure (Event Hub, Service Bus, Cosmos DB) y soporte para múltiples lenguajes.	(disponible en App Platform), con soporte más limitado que los tres grandes pero suficiente para casos de uso básicos.
Redes (VPC)	Amazon VPC (IaaS) Red privada virtual con control total de subnets, tablas de ruteo, ACLs de red y grupos de seguridad.	VPC (IaaS) Red privada virtual global con subneteo regional y políticas de firewall basadas en etiquetas de red.	Azure Virtual Network (IaaS) Red privada con soporte para subnets, tablas de rutas, grupos de seguridad de red (NSG) y peering entre redes.	VPC (IaaS) Redes privadas entre Droplets dentro de un datacenter, con firewall gestionado. Menos granular que los tres grandes, pero suficiente para arquitecturas simples.
Gestión de identidades (IAM)	AWS IAM (SaaS) Gestión de identidades, roles, políticas y permisos granulares por recurso, con soporte para federación de identidad externa.	Cloud IAM (SaaS) Control de acceso basado en roles con integración con Google Workspace y soporte para cuentas de servicio por recurso.	Azure Active Directory / Entra ID (SaaS) Sistema de identidad empresarial con soporte para MFA, SSO, RBAC y federación con Active Directory on-premise.	Access Control (SaaS) Gestión básica de acceso mediante tokens de API y equipos; sin un sistema IAM con roles granulares equivalente al de los tres grandes.
Observabilidad (CloudWatch)	CloudWatch + CloudTrail (PaaS) Monitoreo de métricas, logs, alarmas y trazabilidad de llamadas a la API de AWS.	Cloud Monitoring + Cloud Logging + Cloud Trace (PaaS) Suite integrada de observabilidad con métricas, logs estructurados y trazas distribuidas.	Azure Monitor + Log Analytics + Application Insights (PaaS) Plataforma de observabilidad con métricas, logs, alertas y APM para aplicaciones.	Monitoring (PaaS) Monitoreo básico de métricas de infraestructura (CPU, memoria, red) integrado en el panel de Droplets; para observabilidad avanzada se requieren herramientas de

Capacidad	AWS	GCP	Azure	DigitalOcean
				terceros.

3. Análisis por Dimensiones Técnicas

3.1 Precio y Modelo de Costos

El modelo de precios de los proveedores de nube sigue en todos los casos el esquema de pago por uso (pay-as-you-go), aunque con diferencias relevantes en la granularidad de facturación, la disponibilidad de capa gratuita y la transparencia de los costos indirectos.

AWS ofrece una capa gratuita (Free Tier) con vigencia de 12 meses para nuevas cuentas, que incluye 750 horas/mes de instancias EC2 t2.micro, 5 GB de almacenamiento S3, 20 GB de base de datos RDS, y 1 millón de invocaciones de Lambda por mes de forma permanente (sin vencimiento). Su modelo de precios es el más granular del mercado pero también el más complejo: los costos de transferencia de datos salientes (egress), los costos de peticiones a S3, y los cargos por Multi-AZ en RDS se acumulan de forma poco intuitiva para equipos sin experiencia previa en AWS.

GCP ofrece un crédito inicial de 300 USD para nuevas cuentas (con vigencia de 90 días) y una capa gratuita permanente que incluye 1 instancia e2-micro por mes, 5 GB en Cloud Storage y 2 millones de invocaciones de Cloud Functions. GCP aplica descuentos automáticos por uso sostenido (Sustained Use Discounts) cuando una instancia se utiliza más del 25% de un mes, lo que favorece cargas de trabajo continuas sin necesidad de contratos de reserva.

Azure ofrece 200 USD en crédito para los primeros 30 días y una capa gratuita de 12 meses que incluye VMs B1s (1 vCPU, 1 GB RAM), 5 GB de Blob Storage y 1 millón de ejecuciones de Azure Functions. Azure integra descuentos para organizaciones educativas y estudiantes mediante el programa **Azure for Students**, que otorga 100 USD en crédito sin necesidad de tarjeta de crédito, lo que lo hace especialmente relevante para el contexto universitario de este proyecto.

DigitalOcean no ofrece una capa gratuita permanente, pero proporciona 200 USD en crédito durante 60 días para nuevas cuentas. Su propuesta de diferenciación en precios es la **predictibilidad**: los Droplets tienen precios fijos mensuales (desde 6 USD/mes por 1 vCPU y 1 GB RAM), sin cargos por peticiones de API ni por transferencia de datos salientes hasta un límite incluido en el plan. Esta transparencia es valorada por equipos que necesitan presupuestar con certeza.

Estimación de costo mensual para el caso de estudio (2 instancias de computación pequeñas, 1 base de datos gestionada, 1 bucket de almacenamiento con 50 GB, 1 millón de ejecuciones serverless). Precios estimados aproximados:

Componente	AWS (USD/mes)	GCP (USD/mes)	Azure (USD/mes)	DigitalOcean (USD/mes)
2 instancias de computación	~17 (2× t3.micro)	~14 (2× e2-micro)	~15 (2× B1s)	~12 (2× Droplet básico)

Componente	AWS (USD/mes)	GCP (USD/mes)	Azure (USD/mes)	DigitalOcean (USD/mes)
Base de datos gestionada (PostgreSQL)	~27 (RDS db.t3.micro)	~25 (Cloud SQL micro)	~26 (Flexible Server B1ms)	~15 (Managed DB básico)
Almacenamiento de objetos (50 GB)	~1.15 (S3 Standard)	~1.00 (Cloud Storage)	~1.00 (Blob Hot)	~5.00 (Spaces, plan mínimo)
1M ejecuciones serverless	~0.20 (Lambda)	~0.40 (Cloud Functions)	~0.20 (Azure Functions)	~5.00 (App Platform)
Total estimado	~45	~40	~42	~37

Nota importante: Estas estimaciones son aproximadas y no incluyen costos de transferencia de datos salientes (egress), que en AWS y GCP pueden representar un costo adicional significativo dependiendo del volumen de tráfico. Se recomienda verificar con las calculadoras oficiales de cada proveedor antes de tomar una decisión de producción. Fuente de consulta: calculadoras de precios oficiales de AWS, GCP, Azure y DigitalOcean.

3.2 Seguridad

Los cuatro proveedores evaluados adoptan el **modelo de responsabilidad compartida** (Shared Responsibility Model), aunque con matices en cómo lo documentan y lo hacen operativo para el cliente.

AWS define con claridad que es responsable de la seguridad de la nube (infraestructura física, hipervisores, red subyacente), mientras que el cliente es responsable de la seguridad en la nube (configuración de IAM, grupos de seguridad, cifrado de datos, actualizaciones del SO en instancias EC2). AWS cuenta con las certificaciones ISO 27001, SOC 1/2/3, PCI DSS y HIPAA, y ofrece servicios dedicados a seguridad como AWS Shield (protección DDoS), AWS WAF (firewall de aplicaciones web) y Amazon GuardDuty (detección de amenazas basada en machine learning).

GCP implementa el modelo de responsabilidad compartida con énfasis en la infraestructura de seguridad propia de Google: sus centros de datos utilizan hardware de seguridad personalizado (Titan chips) y cifrado en tránsito y en reposo activado por defecto en todos los servicios, sin costo adicional. GCP es certificado bajo ISO 27001, SOC 1/2/3, PCI DSS y FedRAMP. Su servicio de Cloud Armor ofrece protección DDoS y WAF a nivel de aplicación.

Azure es el proveedor con mayor orientación al mercado corporativo y gubernamental en términos de cumplimiento normativo: cuenta con más de 90 certificaciones de compliance, incluyendo ISO 27001, SOC 1/2/3, HIPAA, FedRAMP High y estándares de cumplimiento regional como ENS (España) o IRAP (Australia). Su integración con **Microsoft Entra ID** (antes Azure Active Directory) facilita la gestión de identidades federadas con infraestructura on-premise existente, lo que lo hace especialmente atractivo para organizaciones con directorio corporativo propio.

DigitalOcean ofrece capacidades de seguridad más básicas: cifrado en reposo para

Volumes y bases de datos gestionadas, grupos de firewall configurables, y certificaciones SOC 2 Tipo II e ISO 27001. No ofrece un sistema IAM con granularidad de roles por recurso comparable al de los tres grandes, lo que puede ser una limitación en entornos con múltiples desarrolladores con distintos niveles de acceso.

Para el caso de UniMarket CUNOC, donde los datos manejados incluyen información de usuarios universitarios y transacciones internas del campus, el cifrado en tránsito (TLS) y en reposo, junto con una política IAM bien definida, son los requisitos mínimos no negociables. Los cuatro proveedores los cubren, aunque con distinto nivel de granularidad y facilidad de configuración.

3.3 Velocidad y Rendimiento

La latencia de red y la disponibilidad de regiones geográficamente próximas a Guatemala son factores relevantes para el rendimiento de UniMarket CUNOC, dado que sus usuarios estarán ubicados en el campus del CUNOC en Quetzaltenango.

AWS cuenta con una región en São Paulo (sa-east-1) como la más cercana a Centroamérica, con latencia promedio desde Guatemala de aproximadamente 80-120 ms. También dispone de puntos de presencia (PoP) de CloudFront (su CDN) en Ciudad de México y Miami, que reducen la latencia para contenido estático a rangos de 20-40 ms.

GCP tiene una región en São Paulo (southamerica-east1) y una en Santiago de Chile (southamerica-west1). Su CDN (Cloud CDN) cuenta con PoPs en Ciudad de México y Miami, con latencias similares a CloudFront para contenido estático. GCP es reconocido en benchmarks independientes por la consistencia de su rendimiento de red, dado que opera sobre la red privada de fibra de Google.

Azure tiene una región en Brasil (brazilsouth) como la más cercana, con latencias similares a las de AWS y GCP desde Guatemala. Su CDN (Azure CDN) también cuenta con PoPs en la región. Adicionalmente, Azure tiene presencia creciente en Latinoamérica con la apertura de regiones en México (2023) y Chile, lo que puede reducir latencias a futuro.

DigitalOcean tiene datacenter en Nueva York y San Francisco como los más cercanos a Guatemala, con latencia promedio de 60-90 ms desde Centroamérica. No dispone de una región en Latinoamérica, lo que puede ser una desventaja para aplicaciones sensibles a latencia.

En cuanto a **cold starts en funciones serverless**, benchmarks públicos comparativos sitúan a AWS Lambda y Azure Functions como los más rápidos en arranque en frío para runtimes comunes como Node.js y Python (rangos de 100-300 ms), mientras que GCP Cloud Functions presenta tiempos similares o ligeramente superiores dependiendo del runtime. Las funciones de DigitalOcean App Platform no están diseñadas como FaaS puro y presentan tiempos de arranque más elevados.

3.4 Experiencia de Usuario para Desarrolladores (DevEx)

La experiencia de desarrollo (Developer Experience o DevEx) es un factor frecuentemente subestimado en la evaluación de proveedores de nube, pero con impacto directo en la velocidad de entrega del equipo.

AWS tiene la consola más funcional pero también la más compleja: la cantidad de servicios

disponibles (más de 200) y la profundidad de sus opciones de configuración generan una curva de aprendizaje significativa para equipos nuevos. Su documentación técnica es extensa y generalmente precisa, y dispone de la comunidad más amplia de desarrolladores con recursos en Stack Overflow, foros oficiales y cursos en AWS Skill Builder.

GCP es valorado positivamente por desarrolladores por la calidad de su consola web y la integración de Cloud Shell (terminal directamente en el navegador sin necesidad de configurar CLI localmente). Su documentación incluye tutoriales interactivos (Codelabs) que facilitan el aprendizaje práctico. La comunidad es más pequeña que la de AWS, pero la calidad de la documentación oficial es consistentemente alta.

Azure tiene la curva de aprendizaje más pronunciada en términos de navegación por su portal, debido a la heterogeneidad de servicios heredados y la reestructuración periódica del portal (Azure Portal). Sin embargo, para equipos familiarizados con el ecosistema Microsoft (.NET, Visual Studio, GitHub), la integración es fluida y la experiencia mejora significativamente. Azure CLI y las extensiones de Visual Studio Code para Azure facilitan el trabajo diario.

DigitalOcean es el proveedor con la mejor experiencia inicial para desarrolladores sin experiencia previa en nube. Su consola es clara y minimalista, su documentación está orientada a tutoriales prácticos paso a paso, y el tiempo hasta el primer despliegue funcional es significativamente menor que en los tres grandes. Esta diferencia fue documentada en el estudio Developer Experience with Cloud Providers (2023), que sitúa a DigitalOcean como el proveedor mejor calificado en facilidad de uso inicial entre desarrolladores junior.

3.5 Integración con Lenguajes de Programación y Frameworks

Lenguaje	AWS Lambda	GCP Cloud Functions	Azure Functions	DigitalOcean Functions
Node.js	Nativo (v18, v20)	Nativo (v18, v20)	Nativo (v18, v20)	Nativo
Python	Nativo (3.10, 3.11, 3.12)	Nativo (3.10, 3.11)	Nativo (3.10, 3.11)	Nativo
Java	Nativo (11, 17, 21)	Nativo (11, 17)	Nativo (8, 11, 17)	No soportado
Go	Nativo (1.21, 1.22)	Nativo (1.21)	Nativo (1.21)	Nativo
.NET / C#	Nativo (.NET 6, 8)	Vía runtime custom	Nativo (.NET 6, 8)	No soportado
Ruby	Nativo (3.2)	Nativo (3.2)	No soportado	No soportado
PHP	Vía runtime custom	Nativo (8.1, 8.2)	Vía runtime custom	No soportado
Runtime custom	(Container Image)	(2nd gen)	(Custom handler)	Limitado

AWS Lambda ofrece el soporte más amplio de runtimes nativos y la posibilidad de desplegar funciones como imágenes de contenedor, eliminando prácticamente cualquier restricción de lenguaje. **Azure Functions** es el más conveniente para equipos .NET. GCP

Cloud Functions de segunda generación amplió significativamente el soporte de runtimes. **DigitalOcean Functions** presenta las limitaciones más relevantes en variedad de lenguajes, lo que puede ser un factor restrictivo dependiendo del stack del equipo.

En cuanto a SDKs, **los cuatro proveedores ofrecen SDKs** oficiales para **Node.js y Python** de calidad comparable. **El SDK de AWS** (aws-sdk / boto3) es el más maduro y con mayor cobertura de servicios. **El SDK de GCP** y el de Azure son igualmente robustos para sus servicios principales. **DigitalOcean** dispone de una librería oficial (do-api) más limitada, y muchos desarrolladores recurren a la API REST directamente o a herramientas como **Terraform** para la gestión de infraestructura.

3.6 Disponibilidad de Herramientas y Ecosistema

AWS tiene el catálogo de servicios más amplio del mercado: más de 200 servicios gestionados que cubren machine learning (SageMaker), IoT (AWS IoT Core), análisis de datos (Redshift, Athena, Kinesis), contenedores (ECS, EKS), registros de contenedores (ECR), Kubernetes gestionado (EKS) y DevOps nativo (CodePipeline, CodeBuild, CodeDeploy). Sin embargo, esta amplitud también implica mayor complejidad de selección y configuración.

GCP es reconocido como líder en servicios de inteligencia artificial y machine learning, con BigQuery para análisis a escala de petabytes, Vertex AI como plataforma unificada de ML, y una red de fibra privada global que favorece transferencias de datos de alta velocidad entre servicios. Su ecosistema de Kubernetes es particularmente fuerte, dado que GKE (Google Kubernetes Engine) fue el primer servicio de Kubernetes gestionado del mercado y sigue siendo referencia.

Azure destaca en su integración con el ecosistema Microsoft: GitHub Actions, Azure DevOps, Visual Studio, Teams y el ecosistema de herramientas empresariales de Microsoft se integran de forma nativa. Para organizaciones que ya utilizan productos Microsoft en su operación diaria, Azure reduce la fricción de integración de forma considerable. Su marketplace incluye soluciones de más de 15,000 proveedores de software independientes.

DigitalOcean tiene un catálogo de servicios más reducido pero suficiente para los casos de uso más comunes: Kubernetes gestionado (DOKS), registro de contenedores, bases de datos gestionadas, espacios de almacenamiento y una plataforma de aplicaciones (App Platform) que simplifica despliegues sin necesidad de gestionar infraestructura directamente. Su marketplace incluye One-Click Apps que permiten desplegar soluciones comunes (WordPress, GitLab, MongoDB) en minutos. Para un proyecto como UniMarket CUNOC, el catálogo de DigitalOcean es suficiente sin ser excesivo.

4. Pruebas de Concepto (PoC)

4.1 Stack Tecnológico Elegido y Justificación

Para los PoCs se seleccionó el siguiente stack tecnológico:

- **Lenguaje:** Node.js 22 (LTS)
- **Framework:** Express.js para la API REST

- **Base de datos:** PostgreSQL (base de datos gestionada en cada proveedor)
- **Contenerización:** Docker (imagen base node:20-alpine)
- **Infraestructura como código:** Terraform para la provisión de recursos (donde aplica)

Justificación: Node.js con Express es un stack ampliamente soportado por los cuatro proveedores evaluados, con SDKs oficiales disponibles en todos los casos. PostgreSQL es el motor de base de datos relacional con mayor soporte en servicios gestionados a través de proveedores. Docker garantiza reproducibilidad del entorno entre proveedores. Esta combinación representa una decisión tecnológica realista para el contexto de UniMarket CUNOC, que ya contempla una arquitectura de microservicios sobre contenedores.

La API desplegada en cada proveedor expone dos endpoints:

1. GET /health retorna: { "status": "ok", "provider": "<nombre>" } para verificar disponibilidad.
2. POST /usuarios: persiste un registro de usuarios simplificado (nombre, correo, password, rol) en la base de datos gestionada y retorna el registro creado.
3. GET /usuarios: recupera todos los registros persistidos.

4.2 PoC en Amazon Web Services (AWS)

Servicios utilizados:

- Computación: ECS (Elastic Container Service)
- Base de datos: Amazon RDS for PostgreSQL (db.t3.micro, Single-AZ)

Descripción del despliegue:

Se desplegó la API del proyecto UniMarket mediante un contenedor Docker en Amazon ECS Express Mode. La imagen fue almacenada en Amazon ECR y ejecutada en ECS con **1 vCPU y 2 GiB de memoria**. Se configuró el puerto 3000 y la conexión con la base de datos PostgreSQL alojada en Amazon RDS.

Dificultades encontradas:

Otra de las dificultades fue la configuración inicial de AWS CLI para poder autenticarse y trabajar con Amazon ECR. También fue necesario subir nuevamente la imagen Docker al registro de AWS, ya que ECS Express Mode utiliza la imagen almacenada en Amazon ECR. Durante el despliegue se presentaron problemas con la comprobación de salud del contenedor, debido a la configuración inicial de la ruta del health check. Fue necesario revisar la configuración del servicio y del balanceador de carga para lograr que la tarea permaneciera en ejecución.

Calidad de la documentación consultada:

La documentación oficial de AWS fue útil para configurar ECS, ECR y RDS, aunque algunos detalles de ECS Express Mode requirieron revisar diferentes secciones de la documentación.

Evidencia de funcionamiento:

Bases de datos > database-unimarket

< database-unimarket

Resumen

Identificador de base de datos database-unimarket CPU 3.90%	Estado ✔ Disponible Clase db.t4g.micro	Rol Instancia Actividad actual 0.00 sesiones	Motor PostgreSQL Región y AZ us-east-1f
--	--	---	--

- Conectividad y seguridad**
- Supervisión
- Registros y eventos
- Configuración
- Integraciones sin extracción, transformación y carga

Conectarse mediante | Información

☒ **Fragmentos de código**
 Úselo cuando se conecte a través de SDK, API o herramientas de terceros, incluidos los agentes.

☐ **CloudShell**
 Úselo para acceder de manera rápida a la CLI de AWS que se lanza de forma rápida desde la Consola de administración de AWS.

☐ **Puertos**
 Úselo para configurar puertos de acceso a la instancia de Amazon RDS.

Reglas del grupo de seguridad (3)	
Filtrar por Reglas del grupo de seguridad	
Grupo de seguridad	Tipo
unimarketcunoc-db-sg (sg-0415a80b891daf387)	CIDR/IP
unimarketcunoc-db-sg (sg-0415a80b891daf387)	CIDR/IP
unimarketcunoc-db-sg (sg-0415a80b891daf387)	CIDR/IP
Replicación (1)	
Tipo	Regla
CIDR/IP - Inbound	190.61.103.127/32
CIDR/IP - Inbound	0.0.0.0/0
CIDR/IP - Outbound	0.0.0.0/0

Amazon Elastic Container Service

Favoritos y recientes

Modo exprés

Clústeres

Espacios de nombres

Definiciones de tareas

Definiciones de tarea de daemon

Configuración de la cuenta

unimarket-poc-515a

Exprés Información

Descripción general del servicio exprés

ARN del servicio
arn:aws:ecs:us-east-1:919441498195:service/default/unimarket-poc-5

Estado del servicio
Activo

Tareas
En ejecución: 1 | En espera: 0 | [Ver tareas](#)

4. Pruebas de petición pos y get hacia la api

POST https://un-c0489b1f6c42432eb3bcd77a638d6af8.ecs.us-east-1.on.aws/usuarios

Body >> JSON Prettify

```
1 {
2   "nombre": "Luis",
3   "correo": "luisbquiaux@gmail.com",
4   "password": "123456",
5   "rol": "administrador"
6 }
```

Response Headers 6 Timeline Tests

```
1 {
2   "id": 2,
3   "nombre": "Luis",
4   "correo": "luisbquiaux@gmail.com",
5   "rol": "administrador",
6   "creadoEn": "2026-09-10T07:03:20.941Z"
7 }
```

GET https://un-c0489b1f6c42432eb3bcd77a638d6af8.ecs.us-east-1.on.aws/usuarios

Params Body >> No Body

No Body

Response Headers 6 Timeline Tests

```
1 [
2   {
3     "id": 1,
4     "nombre": "Luis",
5     "correo": "correo@gmail.com",
6     "rol": "administrador",
7     "creadoEn": "2026-09-10T06:22:03.181Z"
8   }
9 ]
```

4.3 PoC en Google Cloud Platform (GCP)

Servicios utilizados:

- Computación: Google Cloud Run (contenedor serverless)
- Base de datos: Cloud SQL for PostgreSQL (instancia db-f1-micro)

Descripción del despliegue:

Se realizó el despliegue de una API mediante Google Cloud Run, utilizando un contenedor serverless para ejecutar la aplicación. La API se conectó a una instancia de Cloud SQL for PostgreSQL, configurada con el tipo de máquina db-f1-micro, para almacenar y gestionar la información de la aplicación. Durante el proceso se configuraron los servicios necesarios para permitir la comunicación entre el contenedor y la base de datos.

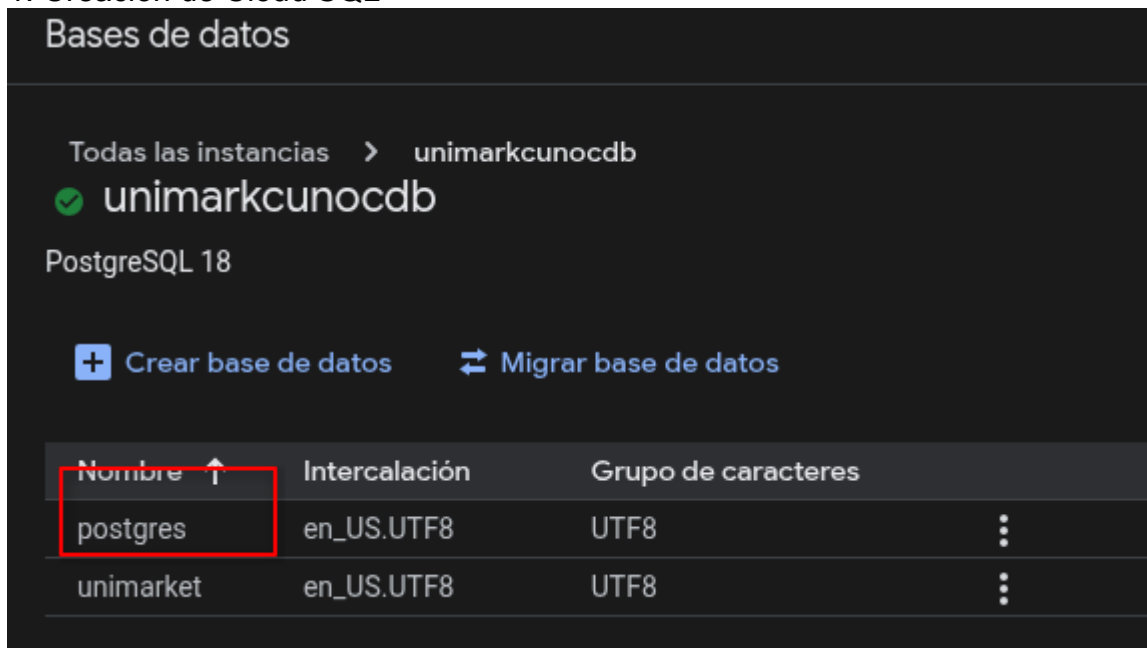
Dificultades encontradas:

Las dificultades encontradas son las siguientes: fue la asignación del rol “Administrador Cloud Run” del usuario IAM, esto impedía que el contenedor donde está desplegado la api (poc) se conectase hacia la instancia de la base de datos. Encontrar específicamente esa dificultad en la documentación oficial, no se detalla específicamente ese caso.

Calidad de la documentación consultada: Media, sin embargo había que leer detenidamente.

Evidencia de funcionamiento:

1. Creación de Cloud SQL



1.1. Asignación de rol al usuario IAM para gestionar Cloud SQL

IAM

Permitir
 Rechazada
 Historial de recomendaciones

Permisos para el proyecto "UnimarketCunoc"

 Estos permisos afectan a este proyecto y todos sus recursos. [Más información](#)

Ver por entidades
 Ver por roles

+ Otorgar acceso
 - Quitar acceso

Filtro
 Escribir el nombre o valor de la propiedad

Tipo	Entidad ↑	Nombre	Rol
☐	404659320761-compute@developer.gserviceaccount.com	Compute Engine default service account	Acceso a tokens de lectura de Developer Connect (Beta) Administrador de Cloud Run Cliente de Cloud SQL Escritor de Artifact Registry Escritor de registros Usuario de cuenta de servicio

2. Configuración de Cloud Platform (servicio creado)

Google Cloud
 UnimarketCunoc

Cloud Run
 ☐

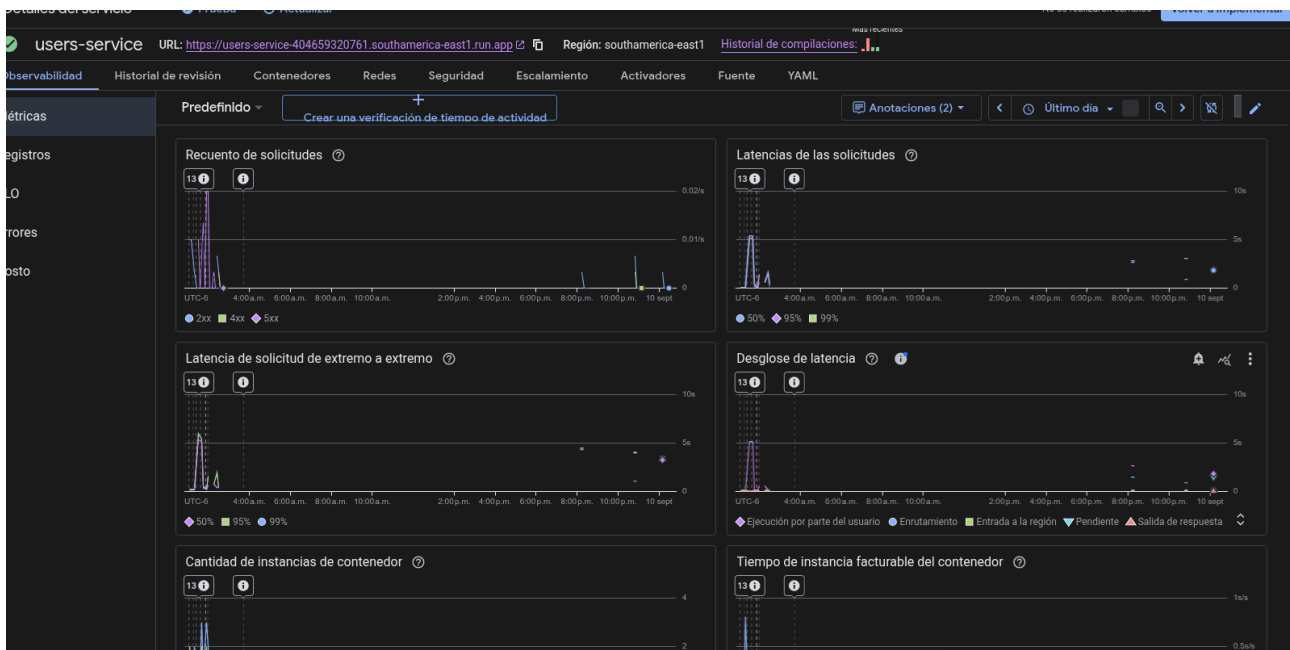
Descripción general
 Servicios
 Trabajos
 Grupos de trabajadores
 Asignaciones de domini...

Servicios
 + Implementar contenedor
 Conectar repo
 (...) Escri...

Cada servicio expone un extremo único y ajusta automáticamente la escala de la infraest...
 Implementa una imagen de contenedor, código fuente o función para crear un servicio.

Filtro
 Filtrar servicios

☐	☐	Nombre ↑	Tipo de implementación	Solicitudes/seg (
☐	☒	users-service	Repository	



3. Pruebas hacia la api

POST registro GET users POST registro gcd GET users gcd POST register users azure GET users azure POST

POST https://users-service-404659320761.southamerica-east1.run.app/usuarios

Params Body * >> JSON Prettify Response Headers 8 Timeline Tests

```
1 {
2   "nombre": "Luis",
3   "correo": "correo@gmail.com",
4   "password": "123456",
5   "rol": "administrador"
6 }
```

```
1 {
2   "error": "El correo ya está registrado."
3 }
```

POST registro GET users POST registro gcd GET users gcd POST register users azure GET users azure POST

GET https://users-service-404659320761.southamerica-east1.run.app/usuarios

Params Body Headers Auth >> Response Headers 8 Timeline Tests

Query

Name	Value	Descri...
Name	Value	Descriptic

Bulk Edit

Path ?

```
1 [
2   {
3     "id": 1,
4     "nombre": "Luis",
5     "correo": "correo@gmail.com",
6     "rol": "administrador",
7     "creadoEn": "2026-09-09T06:01:04.932Z"
8   }
9 ]
```

4. Registro en base de datos:

	nombre	correo	password_hash
1	Luis	correo@gmail.com	\$2b\$10\$VE4j5kUF/PIsicrf0LhvJ.nHEZ6ZfLpIqJWxEtKZ0qdbx

4.4 PoC en Microsoft Azure

Servicios utilizados:

- Computación: Azure Container Apps (Plan F1 gratuito)
- Base de datos: Azure Database for PostgreSQL Flexible Server (Burstable B1ms)

Descripción del despliegue:

Se desplegó la API de UniMarket mediante un contenedor Docker en Azure Container Apps, utilizando una imagen almacenada en Docker Hub. Se configuraron las variables de entorno necesarias para la aplicación y la conexión con Azure Database for PostgreSQL Flexible Server. También se configuraron las reglas de acceso de red de la base de datos para permitir la comunicación con la aplicación.

Dificultades encontradas:

Las principales dificultades fueron la publicación de la imagen en Docker Hub, la configuración de las variables de entorno y las reglas de acceso de red de Azure Database for PostgreSQL. En el primer intento, el despliegue no funcionó debido a que no se habían configurado correctamente las reglas de entrada de la base de datos. Posteriormente, se habilitó el acceso necesario para permitir las peticiones y se logró establecer la conexión correctamente.

Calidad de la documentación consultada:

La documentación oficial de Microsoft Azure fue útil para configurar Azure Container Apps y Azure Database for PostgreSQL. Sin embargo, algunos aspectos relacionados con la configuración de red y las reglas de acceso requirieron revisar varias secciones de la documentación para identificar la configuración necesaria.

Evidencia de funcionamiento:

1. Container Apps configurado

 Buscar

⌵ ⏪

☐ Detener

 Actualizar

 Eliminar

 Envíenos sus comentarios

 Información general

-  Registro de actividad
-  Control de acceso (IAM)
-  Etiquetas
-  Diagnosticar y solucionar problemas
-  Visualizador de recursos
- > Aplicación

 [Now you can manage this application in the Azure Container Apps portal.](#)

^ Essentials

Grupo de recursos [\(mover\)](#)

:

[grupo1](#)

Estado

:

En ejecución

Ubicación [\(mover\)](#)

:

West US 2

Suscripción [\(mover\)](#)

:

[Azure subscription 1](#)

Id. de suscripción

:

c6b7fa3c-f9e2-4cb3-9212-4ad521f22c00

Panel de Aspire

:

Todavía no se ha activado [\(configurar\)](#)

2. Registros de implementación y revisión

Información general

Registro de actividad

Control de acceso (IAM)

Etiquetas

Diagnosticar y solucionar problemas

Visualizador de recursos

Aplicación

Revisiones y réplicas

Contenedores

Escala

Contenedor

Crear nueva revisión

Guardar

Actualizar

Modo de implementación

 [Now you can manage this application in the Azure Container Apps portal.](#)

Cada revisión es una instantánea inmutable de la aplicación contenedora y puede tener d revisión. [Más información](#)

Búsqueda

Revisiones activas

Revisiones inactivas

Réplicas

Nombre ↑ ⌵

Fecha de creación ↓ ⌵

usermarketc--ginje6s

9/9/2026, 8:06:29 p. m.

Reconectar

Detener

Copiar

Borrar

Seleccione los regi minutos después d

Pantalla

Categoría

Réplica

Contenedor

 Reconectar

Conectando

2026-09-10T

2026-09-10T

'usermarket

```
Conectando a la secuencia...
2026-09-10T07:14:37.55749 Connecting to the container 'usermarketc'...
2026-09-10T07:14:37.59715 Successfully Connected to container: 'usermarketc' [Revision: 'usermarketc--gi
'usermarketc--ginje6s-5b9c9c5f6-zpqv1']
2026-09-10T07:14:13.9057645Z stdout F ⬢ injected env (0) from .env // tip: ⬢ encrypted .env [www.dotenvx.
2026-09-10T07:14:14.0630214Z stdout F UniMarket CUNOC PoC escuchando en http://localhost:3000
2026-09-10T07:14:14.0631955Z stdout F GET /health
2026-09-10T07:14:14.0632237Z stdout F POST /usuarios
2026-09-10T07:14:14.0632380Z stdout F GET /usuarios
```

3. Base de datos configurado



Guardar



Descartar



Descargar certificado SSL



Comentarios



Preguntas

- Información general
- Registro de actividad
- Control de acceso (IAM)
- Etiquetas
- Diagnosticar y solucionar problemas
- Visualizador de recursos
- Paneles con Grafana

Conexiones cifradas

TLS/SSL se aplica en el servidor de forma predeterminada. Puede descargar el certificado público c También puede establecer la versión de TLS estableciendo los parámetros del servidor ssl_min_pro

Acceso público

☒ Permitir el acceso público a este recurso a través de Internet mediante una dirección IP públic

Reglas de firewall



Agregar



Eliminar



Actualizar



CLI/PS



Comentarios

Puede crear, ver y eliminar bases de datos PostgreSQL en este servidor. Tenga en cuenta que no puede eliminar ninguna

☐	Nombre ↑	Juego de carac...	Colación	Tipo de esque...	
	azure_maintenance	UTF8	en_US.utf8	System	
	azure_sys	UTF8	en_US.utf8	System	
	postgres	UTF8	en_US.utf8	User	Conectar de
	unimarket	UTF8	en_US.utf8	User	Conectar de

4. Configuración de reglas de seguridad

☒ Permitir acceso público a este servidor desde cualquier servicio de Azure dentro de Azure ⓘ

Agregar dirección IP del cliente actual (190.61.103.127) + Agregar 0.0.0.0 - 255.255.255.255

Nombre de la regla de firewall	Dirección IP
api	
AllowAll_2026-9-9_18-44-13	0.0.0.0
ClientIPAddress_2026-9-9_2-51-22	
ClientIPAddress_2026-9-9_14-42-16	
Nombre de la regla de firewall	Dirección IP i

untos de conexión privados

5. Respues de la peticiones

The screenshot displays a REST client interface with two panels. The top panel shows a POST request to `https://usermarketcrcr.salmoncliff-a6ee62a2.westus2.azurecontainerapps.io/usuarios`. The request body is a JSON object with the following fields: `nombre: "Luis"`, `correo: "correo@gmail.com"`, `password: "123456"`, and `rol: "administrador"`. The response is a JSON object with an `error` field: `"error": "El correo ya está"`.

The bottom panel shows a GET request to the same URL. The response is a JSON array of two user objects. The first object has `id: 1`, `nombre: "Luis"`, `correo: "luisbaquix@gmail.com"`, `rol: "administrador"`, and `creadoEn: "2026-09-09T20:42:56.042Z"`. The second object has `id: 2`, `nombre: "Luis"`, `correo: "correo@gmail.com"`, `rol: "administrador"`, and `creadoEn: "2026-09-10T02:12:43.892Z"`.

```
POST https://usermarketcrcr.salmoncliff-a6ee62a2.westus2.azurecontainerapps.io/usuarios

Body
JSON
Prettify

1 {
2   "nombre": "Luis",
3   "correo": "correo@gmail.com",
4   "password": "123456",
5   "rol": "administrador"
6 }

Response
Headers
Timeline
Tests

1 {
2   "error": "El correo ya está"
3 }
```

```
GET https://usermarketcrcr.salmoncliff-a6ee62a2.westus2.azurecontainerapps.io/usuarios

Params
Body
Headers
Auth
Query

Name Value Descri...
Name Value Descriptic

Bulk Edit

Path

Response
Headers
Timeline
Tests

1 [
2   {
3     "id": 1,
4     "nombre": "Luis",
5     "correo": "luisbaquix@gmail.com",
6     "rol": "administrador",
7     "creadoEn": "2026-09-09T20:42:56.042Z"
8   },
9   {
10    "id": 2,
11    "nombre": "Luis",
12    "correo": "correo@gmail.com",
13    "rol": "administrador",
14    "creadoEn": "2026-09-10T02:12:43.892Z"
15  }
16 ]
```

4.5 PoC en DigitalOcean

Servicios utilizados:

- Computación: App Platform (imagen Docker) Base de datos: Managed Database for PostgreSQL (plan básico)

Descripción del despliegue:

Se desplegó la API de UniMarket mediante App Platform, utilizando una imagen publicada en docker hub. Se creó una base de datos PostgreSQL administrada y se configuraron las

variables de entorno necesarias para la aplicación, incluyendo la cadena de conexión a la base de datos. También fue necesario ajustar la configuración de DB_SSL para establecer correctamente la conexión con PostgreSQL.

Dificultades encontradas:

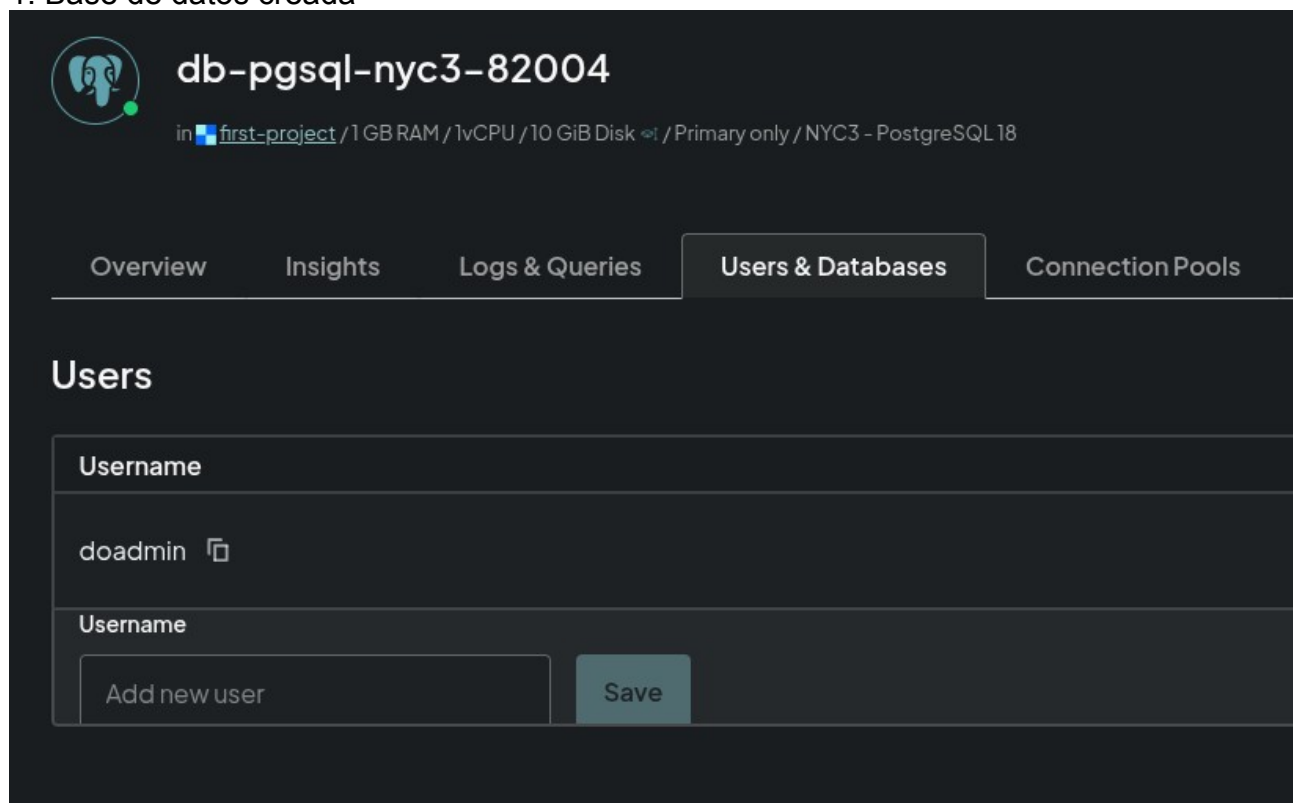
La principal dificultad inicial fue el rechazo de la tarjeta de débito utilizada para registrar el servicio, por lo que fue necesario utilizar otro medio de pago. Durante la configuración también se presentaron dificultades con las variables de entorno y con DB_SSL, que finalmente se dejó vacío para lograr una conexión correcta con la base de datos.

Calidad de la documentación consultada:

La documentación oficial de DigitalOcean fue útil para comprender el proceso de creación de aplicaciones en App Platform y la configuración de bases de datos administradas. La información sobre las variables de entorno y la conexión con PostgreSQL permitió completar la configuración, aunque algunos ajustes requirieron pruebas adicionales.

Evidencia de funcionamiento:

1. Base de datos creada



The screenshot displays the DigitalOcean PostgreSQL database management interface. At the top, the database instance is identified as 'db-pgsql-nyc3-82004' with a PostgreSQL logo icon. Below the instance name, the configuration details are listed: 'in first-project / 1 GB RAM / 1vCPU / 10 GiB Disk / Primary only / NYC3 - PostgreSQL 18'. A navigation bar contains five tabs: 'Overview', 'Insights', 'Logs & Queries', 'Users & Databases' (which is the active tab), and 'Connection Pools'. The main section is titled 'Users' and shows a table with one user, 'doadmin', with a copy icon. Below the table, there is a form to 'Add new user' with a 'Save' button.

Add new user

Save

Databases

Name
defaultdb 
unimarket 

Database Name*

Add new database

Save

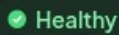
[Support](#) [Status](#) [Docs](#) [Tutorials](#) [Blog](#) [Pricing](#) [Careers](#) [Privacy](#) [Terms](#)

2. App Platform configurada

Apps

 Search for apps

 **clownfish-app**
Deployed about 3 hours ago

 Healthy

PROJECT
 first-project

 1 Web Service

[Live app ↗](#)

Build on what you have

 BUILD WITH AI

 CONTAINER REGISTRY

← Back to Apps

clownfish-app

Healthy • first-project • ATL1 • <https://clownfish-app-78tro.ondigi...>

Overview Insights Activity **Runtime Logs** Console Networking Settings

luisbaquiaux-prac...
Web Service

Time	Message
Sep 18 04:45:45	◇ injected env (8) from .env // tip: % enable debugging { debug: true }
Sep 18 04:45:45	UniMarket CUNOC PoC escuchando en http://localhost:8080
Sep 18 04:45:45	GET /health
Sep 18 04:45:45	POST /usuarios
Sep 18 04:45:45	GET /usuarios
Sep 18 04:45:55	◇ injected env (8) from .env // tip: + encrypted .env [www.dotenvx.com]
Sep 18 04:45:56	UniMarket CUNOC PoC escuchando en http://localhost:8080
Sep 18 04:45:56	GET /health
Sep 18 04:45:56	POST /usuarios
Sep 18 04:45:56	GET /usuarios

3. Respuestas de la api

POST <https://clownfish-app-78tro.ondigitalocean.app/usuarios>

Body • >> **JSON** ▾ Prettify

```
1 {
2   "nombre": "Luis",
3   "correo": "correo@gmail.com",
4   "password": "123456",
5   "rol": "administrador"
6 }
```

Response Headers ¹⁴ Timeline Tests

```
1 {
2   "error": "El correo ya está
3 }
```

GET https://clownfish-app-78tro.ondigitalocean.app/usuarios

Params

Body

Headers

Auth

>>

Query

Name	Value	Descri...	
Name	Value	Descriptic	

Bulk Edit

Path ?

Response

Headers 14

Timeline

Tests

```

1  [
2  {
3      "id": 1,
4      "nombre": "Luis",
5      "correo": "correo@gmail.com",
6      "rol": "administrador",
7      "creadoEn": "2026-09-10T02:45:07.501Z"
8  }
9  ]

```

4.6 Comparativa Empírica

Dimensión	AWS	GCP	Azure	DigitalOcean
Tiempo hasta primer despliegue funcional	Bastante tiempo	Bastante tiempo	Bastante tiempo	1 h
Curva de aprendizaje observada	ALTA	ALTA	ALTA	MEDIA
Dificultades específicas encontradas	Asignar rol IAM a la cuenta de servicio para gestionar imágenes en ECR/ECS; configurar el Security Group de RDS para permitir el puerto 5432; el Target Group del Load Balancer verificaba por defecto la ruta / (no /health), causando terminación forzosa del contenedor (exit 137); migración forzada de App Runner a ECS Express Mode por discontinuación	Confusión entre IP pública y nombre de conexión de Cloud SQL; carácter especial sin codificar en el password (Invalid URL); variable PORT reservada por Cloud Run; permiso IAM Cloud SQL Client faltante (ECONNREFUSED en el socket) pese a tener el conector ya adjunto.	Variable DB_SSL=false arrastrada desde la configuración de GCP, causando rechazo de conexión por falta de cifrado (no encryption); tamaño de cómputo sobredimensionado por defecto en el asistente de creación de la base de datos.	Mismo error DB_SSL=false / no encryption que Azure, por la misma causa raíz, diagnosticado de inmediato al reconocer el patrón ya visto.

Dimensión	AWS	GCP	Azure	DigitalOcean
	del servicio a medio proyecto.			
Costo real vs. costo estimado	Costo estimado: 35 - 45 \$	Costo estimado: 8 - 15 \$	Costo estimado: 0 - 15 \$	Costo estimado: 15 - 20 \$
Fiabilidad durante la prueba	Alta una vez resuelta la ruta del health check; sin caídas posteriores.	Múltiples red despliegues durante la resolución de errores; estable una vez corregido el permiso de IAM.	El primer despliegue fue inestable y se recreó por completo; el segundo, ya estable.	Estable desde el primer diagnóstico correcto; sin incidentes posteriores.
Calidad de documentación consultada	MEDIA	MEDIA	MEDIA	MEDIA

5. Recomendación Fundamentada

5.1 Ventajas y Desventajas por Proveedor

Proveedor	Ventaja principal	Desventaja principal
AWS	Mayor ecosistema de servicios y comunidad más amplia; máxima flexibilidad y cobertura de casos de uso.	Mayor complejidad operativa y curva de aprendizaje más pronunciada; costos difíciles de predecir sin experiencia previa.
GCP	Mejor rendimiento de red y menor latencia en comunicaciones entre servicios; liderazgo en IA/ML.	Comunidad más pequeña que AWS; menor presencia de regiones en Latinoamérica que Azure.
Azure	Mejor integración con ecosistema Microsoft y Active Directory; mayor cobertura de compliance normativo; programa de crédito para estudiantes sin tarjeta de crédito.	Portal más complejo de navegar; documentación menos consistente en servicios no Microsoft.
DigitalOcean	Mayor simplicidad operativa y precios más predecibles; mejor DevEx inicial para equipos sin experiencia en nube.	Menor catálogo de servicios; sin región en Latinoamérica; IAM menos granular.

5.2 Recomendación Técnica

Con base en el análisis documental de las secciones anteriores, y ahora confirmada con la evidencia empírica obtenida de los cuatro PoCs desplegados, la recomendación preliminar se ajusta de la siguiente manera:

5.2.1 Ventajas y desventajas observadas por proveedor

Proveedor	Ventaja principal (evidencia empírica)	Desventaja principal (evidencia empírica)
AWS	Capa gratuita de RDS y EC2/Fargate vigente durante 12 meses completos, la más duradera de los cuatro, relevante si UniMarket CUNOC continuara en operación más allá del curso.	Mayor fricción de último momento: App Runner (el servicio originalmente planeado) dejó de aceptar nuevos clientes durante el desarrollo de esta práctica, obligando a migrar a Amazon ECS Express Mode. Además, es el único proveedor de los cuatro que no acepta imágenes directamente desde Docker Hub, exigiendo duplicar la imagen en Amazon ECR como paso adicional.
Google Cloud (GCP)	Crédito de bienvenida más generoso de los cuatro (USD 300), suficiente para sostener holgadamente todo el ciclo del PoC.	Curva de aprendizaje empíricamente la más pronunciada: se documentaron cinco dificultades distintas durante el despliegue (confusión entre IP pública y nombre de conexión, codificación de caracteres especiales en la cadena de conexión, restricciones de la variable PORT, y un permiso de IAM (Cloud SQL Client) no evidente que impedía la conexión incluso con el conector ya adjunto).
Azure	Flujo de despliegue el más consistente de los cuatro: TLS forzado por defecto (sin necesidad de configurarlo manualmente), asistentes de creación claros, y un crédito de bienvenida (USD 200) más que suficiente para completar el MVP sin costo.	El primer despliegue en Container Apps falló con un error de conexión rechazada por falta de cifrado (no encryption), causado por arrastrar la variable DB_SSL=false desde la configuración usada en GCP (donde sí era correcta) hacia un proveedor que exige TLS obligatorio. La instancia tuvo que recrearse para resolverlo, sin haberse diagnosticado la causa exacta en el momento.
DigitalOcean	La experiencia de desarrollador más simple de las cuatro: acepta imágenes directamente desde Docker Hub sin duplicarlas en un registro propio, y expone las cadenas de conexión ya armadas y listas para copiar, reduciendo errores manuales.	Presentó el mismo error de conexión sin cifrado (no encryption) que Azure, por la misma causa raíz (DB_SSL=false arrastrado incorrectamente desde GCP), aunque en este caso el

Proveedor	Ventaja principal (evidencia empírica)	Desventaja principal (evidencia empírica)
		diagnóstico fue inmediato al reconocer el patrón repetido. Tampoco ofrece una capa gratuita ni un crédito de bienvenida comparable a los otros tres, solo un crédito mínimo de registro (USD 5) y exige método de pago válido desde el inicio.

5.2.2 Contraste entre lo esperado teóricamente y la evidencia práctica

El análisis documental inicial sugería que Google Cloud sería la opción más atractiva. Sin embargo, la evidencia empírica matizó esta conclusión: el criterio de **curva de aprendizaje manejable** (priorizado en segundo lugar) resultó determinante, ya que GCP fue, en la práctica, el proveedor donde más tiempo se invirtió resolviendo problemas de configuración no evidentes desde la documentación oficial (particularmente la relación entre el conector de Cloud SQL y el permiso de IAM Cloud SQL Client, que no está claramente vinculada en una sola guía).

Asimismo, el caso de AWS ilustra un riesgo que el análisis puramente documental no podía anticipar: la disponibilidad de un servicio (App Runner) cambió durante el desarrollo mismo de esta práctica, la arquitectura contenerizada (Docker estándar) permitió absorber ese cambio sustituyendo únicamente el mecanismo de despliegue, sin modificar el código de la aplicación.

5.2.3 Recomendación final

Para el contexto específico de UniMarket CUNOC (un proyecto académico con un ciclo de vida acotado al curso, sin presupuesto operativo definido, y con un equipo sin experiencia previa en nube) se recomienda **Microsoft Azure** como proveedor principal para el despliegue del MVP, por las siguientes razones:

1. **Costo accesible dentro del horizonte del proyecto:** el crédito de bienvenida (USD 200) cubre con holgura el costo de una configuración adecuadamente dimensionada (Azure Database for PostgreSQL Flexible Server en nivel Burstable B1ms + Azure Container Apps).
2. **Curva de aprendizaje manejable:** aunque tanto Azure como DigitalOcean tuvieron una dificultad cada uno, la de Azure (variable DB_SSL mal configurada) fue un error de configuración propio, no una limitación de la plataforma, es decir, atribuible al proceso de despliegue, no al proveedor en sí.
3. **Suficiencia de servicios:** Azure ofrece equivalentes directos para cada componente de la arquitectura de referencia (Container Apps para los microservicios, Azure Database for PostgreSQL para persistencia, Blob Storage para objetos, Azure Functions para tareas serverless, y Azure API Management como API Gateway).

gestionado).

4. Disponibilidad de crédito suficiente para completar el MVP sin incurrir en costos reales durante la duración de la práctica.

Salvedad importante: si UniMarket CUNOC fuera a continuar en operación **más allá del ciclo académico** (es decir, en un escenario de producción real y sostenido), la recomendación cambiaría hacia **AWS**, dado que su capa gratuita tiene una vigencia de 12 meses (frente a los créditos de expiración fija de Azure y GCP), lo cual resulta más adecuado para una operación continua sin presupuesto garantizado.

5.2.4 Arquitectura de despliegue conceptual (Azure)

Componente del sistema	Servicio de Azure propuesto
Microservicios (cómputo)	Azure Container Apps
API Gateway	Azure API Management
Base de datos por servicio	Azure Database for PostgreSQL Flexible Server
Almacenamiento de objetos (imágenes de productos)	Azure Blob Storage
Funciones serverless (notificaciones, validaciones)	Azure Functions
Redes y balanceo de carga	Azure Virtual Network + Azure Application Gateway / Front Door
Identidad y control de acceso	Microsoft Entra ID
Observabilidad (monitoreo, logs, trazas)	Azure Monitor + Application Insights

6. Reflexión Crítica

6.1 Transferibilidad de habilidades entre proveedores

Los conceptos fundamentales de la computación en la nube son transferibles entre proveedores: el modelo de VPC, los grupos de seguridad, el ciclo de vida de una instancia de cómputo, el modelo de almacenamiento de objetos, y los principios de las funciones serverless siguen la misma lógica conceptual independientemente del proveedor. Sin embargo, la transferibilidad práctica es parcial: los nombres de los servicios, las CLI, los modelos de IAM, la sintaxis de los archivos de configuración y los flujos de la consola web son específicos de cada plataforma.

La experiencia con Terraform como herramienta de infraestructura como código es el activo más transferible entre proveedores: los conceptos de provider, resource, state y plan/apply son idénticos, y solo cambia el provider block y los recursos específicos de cada nube. Para un equipo que aspire a trabajar en múltiples proveedores o a comparar alternativas en el futuro, invertir en dominar una herramienta agnóstica de nube como Terraform tiene mayor retorno que especializar profundamente en la CLI propietaria de un único proveedor.

6.2 Riesgos del vendor lock-in

El vendor lock-in o dependencia de proveedor ocurre cuando una aplicación utiliza servicios o características tan específicas de un proveedor que migrar a otro implica un esfuerzo desproporcionado. En el contexto de UniMarket CUNOC, los principales vectores de lock-in identificados son:

1. **Servicios gestionados propietarios:** usar DynamoDB (AWS), Firestore (GCP) o CosmosDB (Azure) en lugar de PostgreSQL estándar crea una dependencia tecnológica que dificultaría la migración, dado que estos motores no tienen equivalente directo en otros proveedores.
2. **Funciones serverless con integraciones nativas:** una Lambda que depende de triggers de SNS, SQS y DynamoDB Streams crea un grafo de dependencias propietario que sería costoso reescribir para otra plataforma.
3. **Servicios de CI/CD nativos:** usar CodePipeline (AWS) o Azure DevOps como pipeline crea dependencia en la plataforma de despliegue; usar GitHub Actions con runners estándar es más portable.

Para mitigar el lock-in en UniMarket CUNOC se recomienda: preferir bases de datos relacionales estándar (PostgreSQL) sobre motores propietarios; usar Docker y Kubernetes como capa de abstracción de cómputo; y definir el pipeline de CI/CD sobre GitHub Actions en lugar de herramientas propietarias del proveedor.

6.3 Estrategia multi-nube o híbrida

Una estrategia **multi-nube** implica distribuir componentes del sistema entre dos o más proveedores de nube, ya sea por resiliencia (evitar un único punto de fallo a nivel de proveedor), por optimización de costos (usar el proveedor más económico para cada servicio), o por cumplimiento normativo (residencia de datos en regiones específicas).

Para UniMarket CUNOC en su etapa actual (MVP universitario), una estrategia multi-nube **no es recomendable**: incrementa significativamente la complejidad operativa, requiere gestionar credenciales, redes y monitoreo en múltiples plataformas simultáneamente, y los beneficios de resiliencia no justifican ese costo operativo para un sistema con los requisitos de disponibilidad de un proyecto académico.

Una estrategia **híbrida** (combinando infraestructura on-premise con servicios en la nube) podría ser relevante si en el futuro el CUNOC dispone de servidores propios en el campus y desea complementarlos con servicios gestionados en la nube para cargas pico o almacenamiento de objetos. En ese escenario, Azure se posicionaría mejor por su integración con Active Directory y su historial en escenarios híbridos empresariales.

6.4 Contraste entre teoría y práctica

La investigación documental inicial, basada únicamente en la comparación de precios, créditos de bienvenida y catálogos de servicios publicados por cada proveedor, sugería que Google Cloud Platform sería la opción más conveniente para este proyecto, principalmente por ofrecer el crédito de bienvenida más alto de los cuatro (USD 300). Bajo ese análisis puramente teórico, la decisión se habría inclinado hacia GCP sin mayor cuestionamiento, ya que ningún documento oficial de precios o de arquitectura permite anticipar cuánto

tiempo y esfuerzo real tomará desplegar un sistema funcional en cada plataforma.

La experiencia práctica de los cuatro PoC contradijo esa expectativa inicial. GCP resultó ser, empíricamente, el proveedor con la curva de aprendizaje más pronunciada de los cuatro: se documentaron cinco dificultades distintas antes de lograr una conexión estable entre el contenedor y la base de datos, incluyendo un permiso de identidad (rol de IAM Cloud SQL Client) que no se menciona de forma evidente en la misma guía donde se explica cómo adjuntar la conexión de Cloud SQL a un servicio de Cloud Run. Ninguna cantidad de lectura previa de documentación hubiera anticipado ese obstáculo específico, porque no es un problema de configuración incorrecta a simple vista, sino de un requisito implícito repartido entre dos secciones distintas de la documentación oficial.

Un segundo contraste relevante surgió al comparar Azure y DigitalOcean. La teoría no distingue entre ambos de forma significativa más allá del precio, pero en la práctica los dos fallaron por exactamente el mismo motivo: la variable de entorno `DB_SSL`, configurada correctamente para GCP (donde la conexión ocurre por un socket local sin cifrado), se reutilizó sin ajustar en ambos proveedores, los cuales exigen TLS de forma obligatoria en cualquier conexión a su base de datos gestionada. Este hallazgo no habría sido visible en un análisis documental, porque no es una limitación de ninguna de las dos plataformas, sino un error propio al asumir que una configuración de seguridad es transferible entre proveedores sin revisión individual.

El caso más contundente de contraste entre teoría y práctica ocurrió con AWS. Ningún análisis documental realizado al inicio de esta práctica podía anticipar que, durante el desarrollo del propio proyecto, Amazon Web Services dejaría de aceptar nuevos clientes en App Runner (el servicio originalmente seleccionado por ser el equivalente directo a Cloud Run y Container Apps) y recomendaría migrar a un servicio de introducción reciente (Amazon ECS Express Mode). Este evento no es atribuible a ningún error de configuración ni a una mala lectura de la documentación; es evidencia directa de que la disponibilidad de un servicio específico de un proveedor de nube no está garantizada en el tiempo, ni siquiera en el corto plazo de un proyecto académico de dos semanas. La decisión de estandarizar el despliegue en una imagen de Docker desde el inicio, en lugar de depender de una API o flujo propietario de un solo servicio, permitió absorber ese cambio sin modificar una sola línea de código de la aplicación, sustituyendo únicamente el mecanismo de orquestación del contenedor.

En conjunto, la experiencia práctica no solo confirmó parcialmente la investigación documental, sino que reveló una categoría de riesgos que ningún análisis teórico puede capturar por completo: la calidad real de la documentación al momento de resolver un problema específico, la fricción de configuración no evidente entre servicios relacionados dentro de una misma plataforma, y la volatilidad misma de la oferta de servicios de un proveedor. Esto lleva a una conclusión metodológica más amplia: una decisión de arquitectura de nube fundamentada únicamente en documentación y tablas comparativas de precios es, en el mejor de los casos, incompleta, y en el peor, puede llevar a subestimar el costo real en tiempo de ingeniería de la opción aparentemente más económica sobre el papel.

7. Conclusiones

Esta práctica demostró que la dificultad real de un despliegue en la nube no siempre coincide con la cantidad de errores documentados. Tanto GCP como Azure tomaron un tiempo considerable de resolución, aun cuando el número de incidentes registrados fue distinto entre ambos: en GCP la guía externa fue determinante para identificar un permiso de IAM no documentado en el mismo lugar que el resto de la configuración, mientras que en Azure el error, a pesar de parecer menor en el papel, tomó varias horas resolverlo.

En ambos casos, contar "número de incidentes" no refleja el costo real en tiempo de ingeniería que representó cada despliegue. AWS, en contraste, resultó el más manejable de los cuatro, aunque esto no fue casualidad: la experiencia previa con la plataforma durante el semestre anterior redujo significativamente la fricción, al punto de no necesitar guía adicional para resolver los ajustes que sí fueron necesarios.

8. Referencias

- [1] Amazon Web Services. AWS Documentation. Recuperado de <https://docs.aws.amazon.com>
- [2] Google Cloud. Google Cloud Documentation. Recuperado de <https://cloud.google.com/docs>
- [3] Microsoft Azure. Azure Documentation. Recuperado de <https://learn.microsoft.com/en-us/azure>
- [4] DigitalOcean. DigitalOcean Documentation. Recuperado de <https://docs.digitalocean.com>
- [5] Mikhail Shilkov. Azure Functions, AWS Lambda, and Google Cloud Functions: Cold starts benchmarks. Recuperado de <https://mikhail.io/serverless/coldstarts/big3/>
- [6] HashiCorp. Terraform Documentation. Recuperado de <https://developer.hashicorp.com/terraform/docs>
- [7] AWS Pricing Calculator. <https://calculator.aws/pricing/2/home>
- [8] Google Cloud Pricing Calculator. <https://cloud.google.com/products/calculator>
- [9] Azure Pricing Calculator. <https://azure.microsoft.com/en-us/pricing/calculator>
- [10] DigitalOcean Pricing. <https://www.digitalocean.com/pricing>